

# App Development Project Report

## App Dev Project Report

### 1. Student Details

**Name:** Ritesh

**Roll Number:**23f3000229

**Email:** [23f3000229@ds.study.iitm.ac.in](mailto:23f3000229@ds.study.iitm.ac.in)

**About Me:** I am a Student at IIT Madras BS Degree program with a deep interest in web application development and data-driven technologies. I enjoy building meaningful applications that combine learning, analytics, and user experience.

---

### 2. Project Details

**Project Title:** Hospital Management App - V2

#### Problem Statement:

You are required to build a **Hospital Management System (HMS)** web application that allows **Admins, Doctors, and Patients** to interact with the system based on their roles.

---

### 3. AI/LLM Declaration

I used **ChatGPT (GPT-5)** to assist in writing SQLAlchemy model definitions, creating API documentation samples, debugging and improving variable naming consistency.

The extent of AI/LLM usage is around **10–15%**, limited to **code suggestions and documentation formatting**.

All final implementation logic, and integration were done manually.

---

### 4. Technologies and Frameworks Used

## Backend

- **Flask 3.1.3** – Main backend web framework used for building the application.
  - **Flask-RESTful** – Used to create RESTful APIs for communication between frontend and backend.
  - **Flask-SQLAlchemy** – ORM used for database interaction and model management.
  - **Flask-Security-Too** – Provides authentication and role-based access control.
  - **SQLAlchemy 2.0.48** – ORM layer for database operations.
  - **Celery 5.6.2** – Used for handling asynchronous background tasks.
  - **Redis 5.0.1** – Used for caching and supporting background task queues.
  - **ReportLab 4.4.10** – Used for generating CSV and PDF reports.
  - **Token-based Authentication** – Used to securely authenticate users and manage API access.
- 

## Frontend

- **Vue.js 3.5.17** – JavaScript framework used for building the user interface.
  - **Vue Router 4.5.1** – Handles navigation and routing between pages in the application.
  - **Vite 7.0.0** – Frontend build tool used for faster development and bundling.
  - **Pinia 3.0.3** – State management library used to manage application data.
  - **Bootstrap 5** – UI framework used for responsive layout and styling.
- 

## Infrastructure

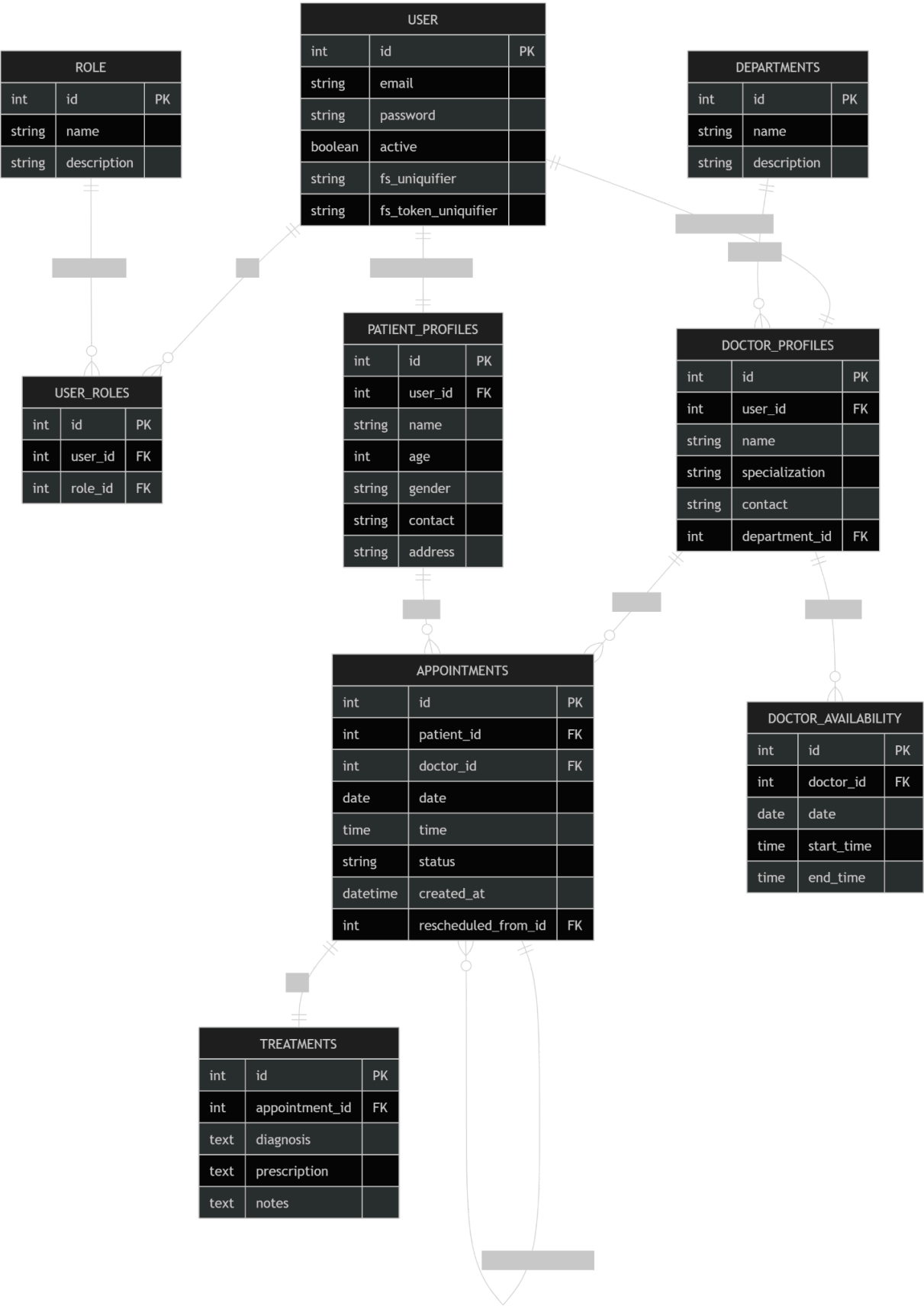
- **Redis** – Used for slot caching with a 10-minute expiration time.
  - **RabbitMQ / AMQP** – Used as the message broker for Celery background tasks.
  - **RESTful API Architecture** – Used for structured communication between frontend and backend services.
- 

## 5. Database Schema / ER Diagram

### Tables:

1. **User** — stores user profile details (id, name, email, password)**User** — stores authentication details of users (id, email, password, active, fs\_uniquifier, fs\_token\_uniquifier).
2. **Role** — stores different system roles for users (id, name, description).
3. **UserRoles** — mapping table that assigns roles to users (id, user\_id, role\_id).
4. **Department** — stores hospital departments such as cardiology, orthopedics, etc. (id, name, description).
5. **DoctorProfile** — stores doctor information including specialization and department (id, user\_id, name, specialization, contact, department\_id).
6. **PatientProfile** — stores patient details such as demographic and contact information (id, user\_id, name, age, gender, contact, address).
7. **Appointment** — stores appointment booking details between doctors and patients (id, patient\_id, doctor\_id, date, time, status, created\_at, rescheduled\_from\_id).
8. **Treatment** — stores diagnosis, prescription, and notes recorded after an appointment (id, appointment\_id, diagnosis, prescription, notes).
9. **DoctorAvailability** — stores the available schedule slots of doctors (id, doctor\_id, date, start\_time, end\_time).

### Relationships:



---

## 6. API Resource Endpoints

### Authentication APIs

These endpoints handle user authentication and account management.

- **POST /api/login** – Authenticates users and allows them to log into the system.
- **POST /api/logout** – Logs out the currently authenticated user.
- **POST /api/register** – Registers a new user account in the system.

---

### Department APIs

These endpoints manage hospital departments. Access is restricted to **admin users only**.

- **GET /api/departments** – Retrieves a list of all departments.
- **POST /api/departments** – Creates a new department.
- **PUT /api/departments/<id>** – Updates an existing department.
- **DELETE /api/departments/<id>** – Deletes a department.

---

### Doctor APIs

These endpoints manage doctor profiles and allow searching and filtering.

- **GET /api/doctors** – Retrieves a list of doctors with search and filtering options.
- **POST /api/doctors** – Creates a new doctor profile.

- **PUT /api/doctors/<id>** – Updates doctor details.
  - **DELETE /api/doctors/<id>** – Deletes a doctor profile.
  - **GET /api/doctors/me** – Retrieves the profile of the currently logged-in doctor.
- 

## Patient APIs

These endpoints manage patient profiles and support filtering and searching.

- **GET /api/patients** – Retrieves a list of patients.
  - **POST /api/patients** – Creates a new patient profile.
  - **PUT /api/patients/<id>** – Updates patient details.
  - **DELETE /api/patients/<id>** – Deletes a patient profile.
  - **GET /api/patients/me** – Retrieves the profile of the currently logged-in patient.
- 

## Appointment APIs

These endpoints handle appointment booking and management.

- **GET /api/appointments** – Retrieves appointment records.
  - **POST /api/appointments** – Books a new appointment.
  - **PUT /api/appointments/<id>** – Updates or reschedules an appointment.
  - **DELETE /api/appointments/<id>** – Cancels an appointment.
- 

## Availability APIs

These endpoints manage doctor availability slots.

- **GET /api/availabilities** – Retrieves available doctor time slots.
  - **POST /api/availabilities** – Adds new availability slots for doctors.
  - **DELETE /api/availabilities** – Removes availability slots.
  - **GET /api/booked-slots** – Retrieves already booked slots (cached for performance).
- 

## Treatment APIs

These endpoints allow doctors to record treatment details for appointments.

- **GET /api/appointments/<id>/treatment** – Retrieves treatment details for a specific appointment.
  - **POST /api/appointments/<id>/treatment** – Adds treatment details for an appointment.
  - **PUT /api/appointments/<id>/treatment** – Updates treatment details.
- 

## Export APIs

These endpoints handle asynchronous export of treatment data.

- **POST /api/export-treatment-csv** – Starts the process of exporting treatment data to CSV.
  - **GET /api/export-status/<task\_id>** – Checks the status of the export task.
  - **GET /api/export-download/<filename>** – Downloads the generated CSV file.
-

## 7. Video Presentation

**Drive Link:**

[https://drive.google.com/drive/folders/1nd\\_oOcX6OWIdx7ew5pSm0aRUBE4LonLi?usp=sharing](https://drive.google.com/drive/folders/1nd_oOcX6OWIdx7ew5pSm0aRUBE4LonLi?usp=sharing)

---